

A Hybrid Learning and Adapting Intrusion Detection in IoT Ecosystems Using Cognitive Federated Reinforcement Framework

A PROJECT REPORT

Submitted by

**KRISHNA CHANDRA G S
MANIBHARATHI VB
DHANESHWAR M**

**RA2211030020002
RA2211030020026
RA2211030020052**

Under the guidance of

Mrs. J. Gowthamy

Assistant Professor

Department of CSE with specialization in Cybersecurity

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

**COMPUTER SCIENCE AND ENGINEERING w/s in
CYBERSECURITY**

of

FACULTY OF ENGINEERING AND TECHNOLOGY



**SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
RAMAPURAM, CHENNAI -600089**

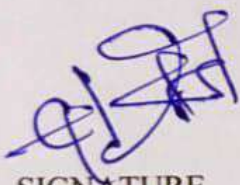
May 2026

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

(Deemed to be University U/S 3 of UGC Act, 1956)

BONAFIDE CERTIFICATE

Certified that this project report titled "A HYBRID LEARNING AND ADAPTING INTRUSION DETECTION IN IOT ECOSYSTEMS USING COGNITIVE FEDERATED REINFORCEMENT FRAMEWORK" is the bonafide work of KRISHNA CHANDRA G S [REG NO: RA2211030020002], V B MANIBHARATHI [REG NO: RA2211030020026], DHANESHWAR M [REGNO: RA2211030020056] who carried out the project work under my supervision. Certified further, that to the best of my knowledge, the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an occasion on this or any other candidate.



SIGNATURE

Mrs. J. Gowthamy, M.E., (Ph.D),

Assistant Professor

Department of CSE with specialization in
Cybersecurity,

SRM Institute of Science and Technology,
Chennai-89.



SIGNATURE

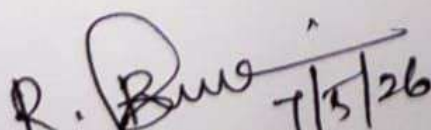
Dr. Shiny Duella J, M.E., Ph.D.,

Associate Professor and Head

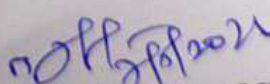
Department of CSE with specialization in
Cybersecurity,

SRM Institute of Science and Technology,
Chennai-89

Submitted for the project viva-voce held on 07-05-2026 at SRM Institute of Science and Technology, Chennai -600089.



7/5/26
INTERNAL EXAMINER



EXTERNAL EXAMINER

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY RAMAPURAM, CHENNAI - 89

DECLARATION

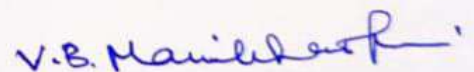
We hereby declare that the entire work contained in this project report titled “ A HYBRID LEARNING AND ADAPTING INTRUSION DETECTION IN IOT ECOSYSTEMS USING COGNITIVE FEDERATED REINFORCEMENT FRAMEWORK” has been carried out by KRISHNA CHANDRA G S [REG NO: RA2211030020026], MANIBHARATHI V [REG NO: RA2211030020026], DHANESHWAR M [REG NO: RA2211030020026] at SRM Institute of Science and Technology, Ramapuram, Chennai- 600089, under the guidance of Mrs, J. Gowthamy, M.E, (Ph.D)., Assistant Professor , Department of CSE in specialization with Cybersecurity.

Place: Chennai

Date: 07-05-2026



KRISHNA CHANDRA G S



MANIBHARATHI VB



DHANESHWAR M



Own Work Declaration

Department of Computer Science and Engineering w/s in Cybersecurity

**SRM Institute of Science and Technology
Own Work Declaration form**

Degree/ Course : B. Tech Computer Science and Engineering w/s in Cybersecurity
Student Name : KRISHNA CHANDRA GS, MANIBHARATHI V, DHANESHWAR M
Registration Number : RA2211030020002, RA2211030020026, RA2211030020052
Title of Work : A Hybrid Learning and Adapting Intrusion Detection in IoT Ecosystems Using
Cognitive Federated Reinforcement Framework

We hereby certify that this assessment complies with the University's Rules and Regulations relating to Academic misconduct and plagiarism, as listed in the University Website, Regulations, and the Education Committee guidelines.

We confirm that all the work contained in this assessment is our own except where indicated, and that We have met the following conditions:

- Clearly references / listed all sources as appropriate
- Referenced and put in inverted commas all quoted text (from books, web, etc)
- Given the sources of all pictures, data etc. that are not our own
- Not made any use of the report(s) or essay(s) of any other student(s) either past or present
- Acknowledged in appropriate places any help that we have received from others (e.g., fellow students, technicians, statisticians, external sources)
- Compiled with any other plagiarism criteria specified in the Course handbook / University website

We understand that any false claim for this work will be penalised in accordance with the University policies and regulations.

DECLARATION:

We are aware of and understand the University's policy on Academic misconduct and plagiarism and we certify that this assessment is our own work, except where indicated by referring, and that we have followed the good academic practices noted above.

RA2211030020002

RA2211030020026

RA2211030020052

ACKNOWLEDGEMENT

We thank the Almighty for giving us the strength and guidance to successfully complete this project. We place on record our deep sense of gratitude to our honourable Chairman **Dr. R. SHIVAKUMAR, MBBS., MD.,** for providing us with the requisite infrastructure throughout the course.

We take the opportunity to extend our hearty and sincere thanks to our **Dean, Dr. M. SAKTHI GANESH., Ph.D.,** for guiding us towards the successful completion of the project.

We take the privilege to extend our hearty and sincere gratitude to the Professor and Head of the department, **Dr. SHINY DUELA J, M.E., Ph.D.,** for her suggestions, support and encouragement towards the completion of the project with perfection.

We express our hearty and sincere thanks to our guide **Mrs. J. GOWTHAMY, M.E., (Ph.D.),** Assistant Professor, Department of CSE with specialization in Cybersecurity for her encouragement, motivation and constant guidance throughout this project work.

Our thanks to the teaching and non-teaching staff of the Department of Cybersecurity of SRM Institute of Science and Technology, Chennai, for providing necessary resources for our project.

We also thank our team members for their cooperation, coordination, and support throughout the project. We extend our gratitude to our parents and family members for their continuous support and encouragement.

ABSTRACT

The rapid growth of Internet of Things (IoT) technology has introduced significant security challenges due to the increasing number of interconnected devices and the dynamic nature of network environments. Ensuring secure communication and detecting malicious activities in such distributed systems has become a critical concern in the field of cybersecurity. This project presents an effective approach for intrusion detection in IoT networks using a deep learning-based model combined with intelligent and adaptive techniques. Unlike traditional intrusion detection systems that rely on predefined rules and signature-based methods, the proposed system utilizes a Deep Neural Network (DNN) to automatically learn patterns from network traffic data and classify activities as normal or malicious. The system is designed with a structured pipeline that includes data preprocessing, feature extraction, model training, and classification. The trained model is capable of identifying complex attack patterns and detecting both known and unknown threats with improved accuracy. Furthermore, the proposed framework incorporates the concepts of federated learning and reinforcement learning to enhance scalability, privacy preservation, and adaptability. The decentralized learning approach ensures that sensitive data is not shared between devices, while adaptive decision-making mechanisms improve the system's ability to respond to evolving cyber threats. This intrusion detection system offers significant advantages in securing IoT environments by reducing false positives, improving detection efficiency, and supporting real-time threat monitoring in critical applications.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
	ABSTRACT	vi
	LIST OF FIGURES	ix
	LIST OF ACRONYMS AND ABBREVIATIONS	x
1	INTRODUCTION	1
	1.1 Problem Statement	1
	1.2 Aim of the Project	2
	1.3 Project Domain	3
	1.4 Scope of the Project	3
	1.5 Methodology	3
	1.6 Organization of the Report	4
2	LITERATURE REVIEW	6
3	PROJECT DESCRIPTION	8
	3.1 Existing System	8
	3.2 Proposed System	8
	3.2.1 Advantages	9
	3.3 Feasibility Study	9
	3.3.1 Economic Feasibility	9
	3.3.2 Technical Feasibility	10
	3.3.3 Social Feasibility	10
	3.4 System Specification	10
	3.4.1 Hardware Specification	10
	3.4.2 Software Specification	10
4	PROPOSED WORK	11
	4.1 General Architecture	11
	4.2 Design Phase	12
	4.2.1 Data Flow Diagram	12
	4.2.2 UML Diagram	15
	4.2.3 Use Case Diagram	16
	4.2.4 Sequence Diagram	17
	4.3 Module Description	17

	4.3.1 Module 1: Data Processing	17
	4.3.2 Module 2: Feature Extraction	18
	4.3.3 Module 3: Intrusion Detection Using DNN	18
	4.3.4 Step 2: Processing of Data	18
	4.3.5 Step 3: Split the Data	19
	4.3.6 Dataset Sample	19
	4.3.7 Step 4: Building the Model	19
	4.3.8 Step 5: Compiling and Training the Model	20
5	IMPLEMENTATION AND TESTING	22
	5.1 Input and Output	22
	5.1.1 Input data (network traffic)	22
	5.1.2 Predicted output (intrusion detection result)	22
	5.2 Testing	22
	5.2.1 Types of Testing	23
	5.2.2 Unit testing	23
	5.2.3 Blackbox Testing	24
	5.2.4 Integration testing	24
	5.2.5 Functional testing	26
	5.2.6 Sanity Testing	26
	5.2.7 Regression Testing	27
	5.2.8 Performance Testing	28
	5.2.9 Test Result	29
6	RESULTS AND DISCUSSIONS	30
	6.1 Efficiency of the Proposed System	30
	6.2 Comparison of Existing and Proposed System	30
7	CONCLUSION AND FUTURE ENHANCEMENTS	31
	7.1 Conclusion	31
	7.2 Future Enhancements	31
	REFERENCES	32

APPENDIXES

A. Sample source code

B. Screenshots

C. Proof of publications

LIST OF FIGURES

Fig No.	Figure Name	Page No.
4.1	Architecture Diagram	11
4.2	Data Flow Diagram Level 0	12
4.3	Data Flow Diagram Level 1	13
4.4	Uml Diagram	15
4.5	Use Case Diagram	16
4.6	Sequence Diagram	17
4.7	Pre-Processing Of Data	18
4.8	Sample of Normal Network Traffic	19
4.9	Sample of Malicious Network Traffic	19
4.10	Model Summary	20
4.11	Code for Model Compilation and Training	21
5.1	Sample Input Network Traffic Data	22
5.2	Predicted Intrusion Detection Output	22
5.3	Data Preprocessing Code Using Python Libraries	23
5.4	Code for Loading and Displaying TEST RESULT	25
5.5	New Model Summary	25
5.6	Training and Testing Data Split	26
5.7	Model Compilation and Training Process	26
5.8	Performance Testing	28

LIST OF ACRONYMS AND ABBREVIATIONS

Acronym	Full Form
API	Application Programming Interface
IDS	Intrusion Detection System
IoT	Internet of Things
IIoT	Industrial Internet of Things
AI	Artificial Intelligence
ML	Machine Learning
DL	Deep Learning
CNN	Convolutional Neural Network
LSTM	Long Short-Term Memory
RL	Reinforcement Learning
DQN	Deep Q-Network
FL	Federated Learning
SVM	Support Vector Machine
k-NN	k-Nearest Neighbors
DBSCAN	Density-Based Spatial Clustering of Applications with Noise
SIEM	Security Information and Event Management
DoS	Denial of Service
DDoS	Distributed Denial of Service

CHAPTER 1

INTRODUCTION

Intrusion detection in Internet of Things (IoT) environments has become a critical aspect of modern cybersecurity due to the rapid increase in connected devices and network complexity. IoT systems are widely used across various domains such as healthcare, smart homes, industrial automation, and transportation, making them highly attractive targets for cyber-attacks. Ensuring secure communication and protecting sensitive data in such distributed environments is essential for maintaining system reliability and user trust. Traditional intrusion detection systems rely on signature-based or rule-based approaches, which are often ineffective in detecting new and sophisticated attacks. To overcome these limitations, machine learning and deep learning techniques have been introduced to analyze network traffic and identify malicious activities. These approaches enable the system to learn patterns from historical data and improve detection accuracy. In particular, deep learning models such as Deep Neural Networks (DNN) are capable of capturing complex relationships in high-dimensional data, making them suitable for intrusion detection tasks. Intrusion detection in IoT networks involves analyzing large volumes of heterogeneous data generated by multiple devices. The challenge lies in identifying abnormal patterns within this data while maintaining efficiency and scalability. Data preprocessing and feature extraction play a crucial role in improving the performance of detection models. By transforming raw network traffic into structured features, the system can effectively distinguish between normal and malicious behavior. Overall, intrusion detection systems based on intelligent learning techniques provide an efficient solution for securing IoT environments. They not only enhance detection accuracy but also support real-time monitoring and response, making them suitable for critical applications. The development of such systems is essential for addressing the evolving cybersecurity challenges in modern IoT ecosystems.

1.1 PROBLEM STATEMENT

Network traffic patterns, data flow behavior, and communication anomalies play a crucial role in identifying cyber threats within Internet of Things (IoT) environments. With the rapid growth of interconnected devices, ensuring secure communication and detecting malicious activities has become increasingly complex. Traditional intrusion detection systems are primarily designed for conventional

networks and often fail to handle the dynamic and heterogeneous nature of IoT systems. These systems struggle to detect advanced and previously unseen attacks due to their reliance on static rules and predefined signatures. Intrusion detection in IoT networks presents several challenges, including high data volume, diverse communication protocols, and resource-constrained devices. The presence of both normal and malicious traffic with overlapping characteristics makes classification a difficult task. Additionally, centralized detection approaches introduce privacy concerns and scalability limitations, as sensitive data must be transmitted and processed at a central location.

Existing machine learning-based intrusion detection methods improve detection accuracy but often lack adaptability to evolving attack patterns. Deep learning models, although more powerful, require efficient feature extraction and optimized architectures to perform effectively in real-time environments. Therefore, there is a need for an intelligent intrusion detection system that can accurately identify threats, minimize false alarms, and operate efficiently in distributed IoT networks. The problem addressed in this project is to design a robust and scalable intrusion detection system that can analyze network traffic, detect both known and unknown attacks, and adapt to changing threat patterns while ensuring data privacy and computational efficiency.

1.2 AIM OF THE PROJECT

- To develop an intelligent intrusion detection system capable of accurately identifying malicious activities in IoT network traffic using deep learning techniques.
- To improve the model's ability to generalize across diverse IoT environments by adapting to varying network patterns and data distributions.
- To enhance feature extraction mechanisms for analyzing complex and high-dimensional network traffic data, enabling precise classification of normal and attack behavior.
- To implement a scalable and efficient learning framework that can operate effectively in resource-constrained IoT environments with minimal computational overhead.

To incorporate privacy-preserving and adaptive learning techniques, such as federated learning and reinforcement learning, to improve system performance, adaptability, and real-time threat detection.

1.3 PROJECT DOMAIN

The domain of this project lies in Cybersecurity and Internet of Things (IoT), with a primary focus on Deep Learning (DL) techniques for network intrusion detection. It emphasizes the application of machine learning and deep learning models, particularly Deep Neural Networks (DNN), for analyzing network traffic and identifying malicious activities. The project also incorporates concepts from distributed learning and intelligent systems, including federated learning and reinforcement learning, to enhance scalability, adaptability, and data privacy in IoT environments.

1.4 SCOPE OF THE PROJECT

The scope of this project includes the design and implementation of an intelligent intrusion detection system for Internet of Things (IoT) environments using deep learning techniques. The system incorporates modules for data collection, preprocessing, feature extraction, and classification of network traffic into normal and malicious categories. It leverages Deep Neural Networks (DNN) to learn complex patterns in network data and improve detection accuracy.

The project also explores the integration of advanced learning concepts such as federated learning and reinforcement learning to enhance system scalability, adaptability, and data privacy. It aims to address key challenges such as handling high-dimensional network traffic, detecting unknown attack patterns, and operating efficiently in resource-constrained IoT devices. Additionally, the system focuses on minimizing false positives and ensuring reliable real-time intrusion detection, making it suitable for practical deployment in modern IoT applications.

1.5 METHODOLOGY

In this project, a deep learning-based approach is adopted to develop an efficient intrusion detection system for IoT environments. The proposed framework utilizes a Deep Neural Network (DNN) model to analyze network traffic data and accurately classify it into normal and malicious categories. The methodology focuses on capturing complex patterns in network behavior and addressing challenges such as high-dimensional data, dynamic traffic patterns, and evolving cyber threats. The system is designed with multiple processing stages to ensure effective detection. Initially, raw network traffic data is collected and subjected to preprocessing techniques such as data cleaning, normalization, and transformation into structured formats. This step improves data quality and ensures that the model can learn meaningful

patterns. Following preprocessing, feature extraction is performed to identify significant attributes that represent network behavior. These features are used as input to the deep learning model for training and classification.

The core component of the system is the Deep Neural Network, which consists of multiple hidden layers that enable the model to learn non-linear relationships within the data. The model is trained using labeled datasets to distinguish between normal and attack traffic. Additionally, concepts such as federated learning are considered to support decentralized model training, while reinforcement learning is introduced to enhance adaptability by enabling the system to adjust its detection strategy based on feedback. The overall methodology is structured into three main modules: Data Preprocessing, Feature Extraction, and Intrusion Detection using Deep Learning. Each module plays a vital role in ensuring accurate and efficient detection. Through this integrated approach, the project aims to provide a scalable, adaptive, and reliable solution for securing IoT networks against cyber threats.

1.6 ORGANIZATION OF THE REPORT

Chapter 2 presents a comprehensive literature review of existing research related to intrusion detection systems in IoT environments. It discusses various machine learning and deep learning approaches, highlighting their strengths and limitations. The chapter also identifies gaps in existing methods, which form the basis for the proposed system.

Chapter 3 focuses on the existing system and its limitations in detecting intrusions within IoT networks. It explains the challenges associated with traditional and AI-based intrusion detection systems, such as scalability, data privacy concerns, and computational constraints. The chapter also introduces the proposed system, which utilizes deep learning techniques for improved detection performance. Additionally, it includes a feasibility study covering economic, technical, and social aspects, along with system specifications required for implementation.

Chapter 4 describes the proposed work in detail. It begins with the overall system architecture, illustrating how data flows through different modules of the intrusion detection system. The design phase includes diagrams such as Data Flow Diagram, UML Diagram, Use Case Diagram, and Sequence Diagram to represent system functionality and interactions. The chapter further explains the core modules, including data preprocessing, feature extraction, and intrusion detection using a Deep Neural Network. It also outlines the steps involved in data processing, model building, and training.

Chapter 5 covers the implementation and testing of the proposed system. It describes the input and output of the system, where network traffic data is processed to generate intrusion detection results. Various testing methods such as Unit Testing, Integration Testing, and Functional Testing are performed to ensure system reliability and accuracy. The chapter demonstrates how the model is trained and evaluated using real datasets.

Chapter 6 presents the results and discussion of the proposed system. It evaluates the efficiency and performance of the model based on metrics such as accuracy and detection rate. The chapter also compares the proposed system with existing approaches, highlighting improvements in detection capability, scalability, and adaptability. The results demonstrate the effectiveness of the system in identifying cyber threats in IoT environments.

Chapter 7 provides the conclusion and future enhancements of the project. It summarizes the key contributions of the proposed intrusion detection system and its effectiveness in improving IoT security. The chapter also outlines potential future improvements, such as integrating advanced learning techniques, improving model robustness, and deploying the system in real-world scenarios.

Chapter 8 includes the source code and additional implementation details. It provides sample code snippets, tools used, and guidelines for reproducing the project. This chapter also contains supporting materials such as screenshots and presentation details related to the project.

CHAPTER 2

LITERATURE REVIEW

This chapter presents a comprehensive review of existing research and scholarly work related to intrusion detection systems in Internet of Things (IoT) environments. The objective is to highlight key advancements, methodologies, and limitations in the field. By analyzing various research contributions, this section provides a clear understanding of current technologies and identifies gaps that motivate the proposed system.

M. Muzammul et al. [1] explored the application of advanced machine learning techniques in IoT-based intrusion detection systems. Their work focused on improving detection accuracy using multi-scale data analysis and intelligent feature extraction. The study demonstrated strong performance in identifying complex attack patterns. However, the approach required high computational resources, making it less suitable for deployment in resource-constrained IoT devices.

N. D. Vo et al. [2] examined data-driven approaches for analyzing large-scale network environments. Their research emphasized the importance of handling heterogeneous data sources and improving system adaptability. While the proposed methods enhanced data understanding, challenges remained in terms of scalability and real-time processing in dynamic IoT networks.

W. Lu et al. [3] proposed a hybrid deep learning model combining convolutional neural networks and transformer-based architectures. The model effectively captured both local and global features from data, resulting in improved detection performance. Despite its advantages, the complexity of the model increased computational overhead, limiting its practical implementation in lightweight IoT systems.

L.-Q. Zhou et al. [4] introduced an improved detection framework using advanced convolutional techniques for identifying anomalies in network traffic. Their approach enhanced feature representation and classification accuracy. However, the system showed limitations in handling highly dynamic and evolving attack patterns.

Y. Tao et al. [5] focused on developing lightweight intrusion detection models suitable for real-time applications. Their study emphasized reducing model size while maintaining detection efficiency.

Although the approach improved processing speed, it faced trade-offs in detection accuracy, especially for complex attacks.

J. Zhang et al. [6] proposed a multi-scale feature fusion approach to enhance detection performance in distributed environments. By integrating features from multiple layers, the system achieved better classification results. However, the increased complexity of feature fusion introduced additional processing overhead.

C. Liu et al. [7] developed a lightweight deep learning model optimized for edge devices. Their work demonstrated that efficient models can achieve competitive performance while minimizing resource consumption. Nevertheless, the model struggled with detecting rare and previously unseen attack patterns.

D. Padilla Carrasco et al. [8] investigated multi-scale detection techniques for improving accuracy in complex environments. Their findings highlighted the importance of selecting appropriate feature scales for accurate detection. However, the approach required careful parameter tuning to achieve optimal performance.

X. Qian et al. [9] introduced a lightweight feature fusion network designed to improve detection efficiency while maintaining accuracy. The model showed promising results in real-time scenarios but faced challenges in handling high-dimensional data effectively.

Overall, the reviewed literature demonstrates that significant progress has been made in applying machine learning and deep learning techniques to intrusion detection in IoT environments. However, key challenges such as scalability, adaptability, privacy preservation, and real-time performance remain unresolved. These limitations highlight the need for a hybrid approach that integrates deep learning with advanced techniques like federated learning and reinforcement learning. The proposed system in this project aims to address these challenges by providing a scalable, adaptive, and efficient intrusion detection solution.

CHAPTER 3

PROJECT DESCRIPTION

3.1 EXISTING SYSTEM

Existing intrusion detection systems in IoT environments primarily rely on traditional security mechanisms such as signature-based and rule-based approaches. These systems are designed to detect known threats by matching network traffic patterns with predefined signatures. However, due to the rapidly evolving nature of cyber-attacks, these approaches often fail to identify new and sophisticated threats, including zero-day attacks.

Recent advancements have introduced machine learning and deep learning-based intrusion detection systems to improve detection accuracy. These systems analyze network traffic data and learn patterns associated with normal and malicious behavior. Techniques such as Support Vector Machines (SVM), Decision Trees, and basic neural networks have been widely used for classification tasks. While these approaches show improved performance compared to traditional systems, they still face limitations in handling large-scale and heterogeneous IoT data.

Another challenge in existing systems is the dependency on centralized data processing, where all network data is collected and analyzed at a single location. This creates issues related to data privacy, communication overhead, and scalability. Additionally, many models require high computational resources, making them unsuitable for deployment on resource-constrained IoT devices.

The limitations of existing systems highlight the need for more intelligent, scalable, and adaptive intrusion detection mechanisms that can effectively operate in distributed IoT environments while maintaining high accuracy and low false positive rates.

3.2 PROPOSED SYSTEM

The proposed system introduces an advanced intrusion detection framework for IoT environments using deep learning and adaptive learning techniques. The system is designed to accurately classify network traffic into normal and malicious categories by leveraging the capabilities of Deep Neural Networks (DNN).

The framework incorporates a structured pipeline consisting of data preprocessing, feature

extraction, and classification. The DNN model learns complex patterns and relationships within network traffic data, enabling it to detect both known and unknown attacks effectively. To address privacy concerns and scalability challenges, the system integrates federated learning, allowing multiple devices to collaboratively train the model without sharing raw data.

Additionally, reinforcement learning is incorporated to enhance adaptability. This allows the system to dynamically adjust its detection strategies based on changing network conditions and evolving cyber threats. The combination of deep learning, federated learning, and reinforcement learning ensures improved detection accuracy, reduced false positives, and efficient real-time performance.

The proposed approach provides a scalable and intelligent solution for securing IoT networks while overcoming the limitations of traditional intrusion detection systems.

3.2.1 ADVANTAGES

- Improves detection accuracy for complex and high-dimensional network data
- Reduces computational overhead through optimized learning techniques
- Enhances scalability for large and distributed IoT environments
- Capable of identifying both known and unknown cyber threats
- Preserves data privacy using decentralized learning approaches
- Supports real-time intrusion detection with improved efficiency

3.3 FEASIBILITY STUDY

A feasibility study is conducted to evaluate the practicality and effectiveness of the proposed system. This analysis helps in understanding whether the system can be successfully implemented in real-world environments. The feasibility is assessed based on three major aspects:

- Economic Feasibility
- Technical Feasibility
- Social Feasibility

3.3.1 ECONOMIC FEASIBILITY

The system is developed using widely available technologies such as Python, Jupyter Notebook, and machine learning libraries. These tools are open-source and well-supported, ensuring ease of implementation. The required technical skills are achievable for students and professionals with basic knowledge of programming and machine learning, making the system technically feasible.

3.3.2 TECHNICAL FEASIBILITY

The system is developed using widely available technologies such as Python, Jupyter Notebook, and machine learning libraries. These tools are open-source and well-supported, ensuring ease of implementation. The required technical skills are achievable for students and professionals with basic knowledge of programming and machine learning, making the system technically feasible.

3.3.3 SOCIAL FEASIBILITY

The proposed intrusion detection system contributes positively to society by enhancing the security of IoT environments. It helps in protecting sensitive data, preventing cyber-attacks, and ensuring reliable system operation. The system is user-friendly and does not require complex interactions, making it suitable for widespread adoption in various domains such as healthcare, smart homes, and industrial systems.

3.4 SYSTEM SPECIFICATION

An efficient system setup is essential for the successful execution of machine learning and deep learning tasks. Proper hardware and software configurations ensure smooth processing, faster computations, and accurate model performance.

3.4.1 HARDWARE SPECIFICATION

- Processor: Intel i5 / i7 / i9 (10th Generation or higher)
- Network: Ethernet connection or Wi-Fi adapter
- Storage: Minimum 100 GB (Recommended 200 GB or more)
- Memory(RAM): Minimum 8 GB (Recommended 16 GB or above)

3.4.2 SOFTWARE SPECIFICATION

- Python
- Anaconda
- Jupyter Notebook
- TensorFlow
- Keras
- Scikit-learn
- pandas
- matplotlib

CHAPTER 4

PROPOSED WORK

4.1 GENERAL ARCHITECTURE

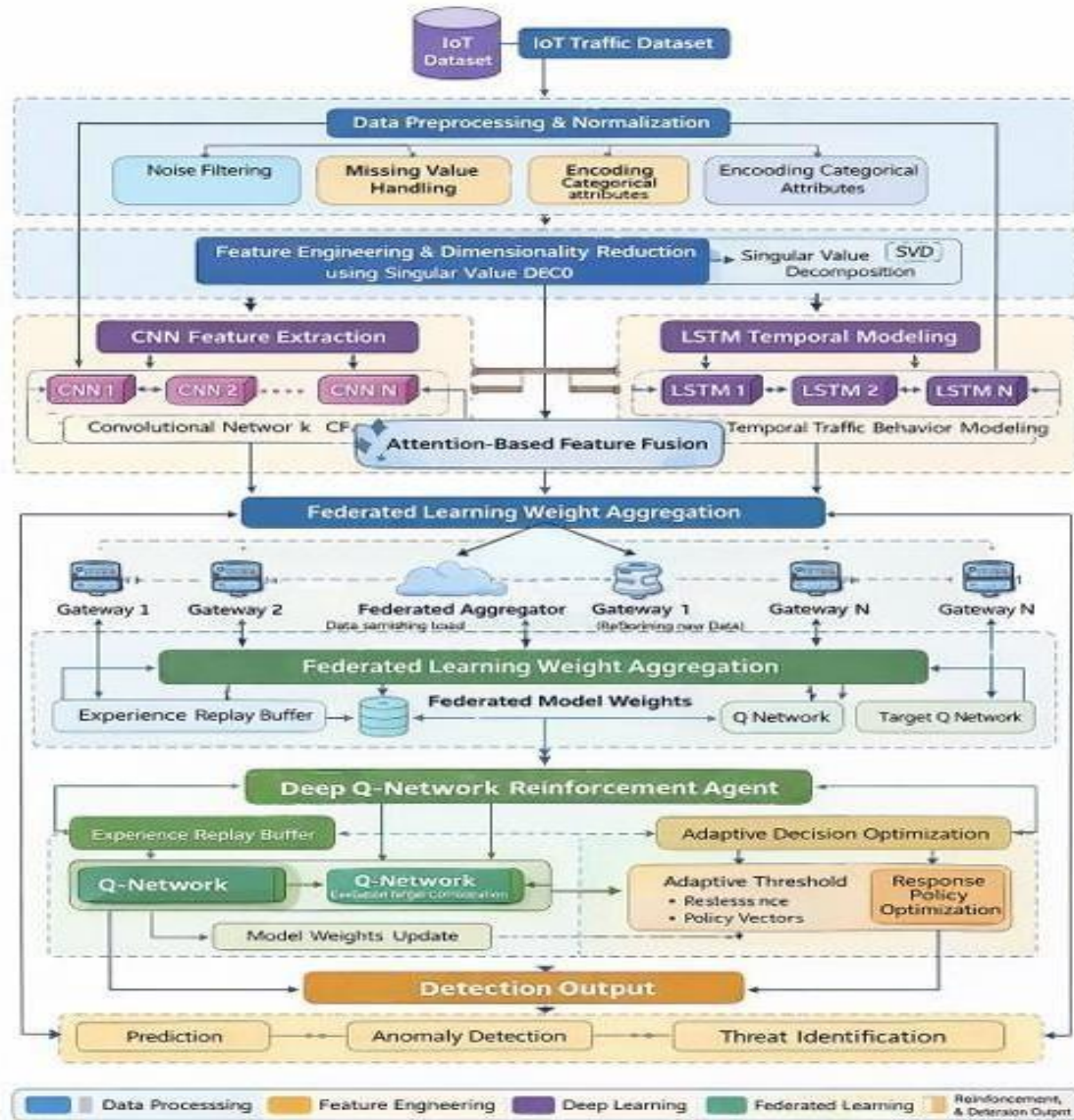


Figure 4.1: Architecture Diagram

Figure 4.1 illustrates the overall architecture of the proposed intrusion detection system designed for IoT environments.

The architecture is based on a deep learning framework that integrates data preprocessing, feature extraction, and classification modules. Network traffic data is collected from IoT devices and passed through preprocessing stages to remove noise and normalize the data. The processed data is then fed into a Deep Neural Network (DNN) model, which learns patterns of normal and malicious activities. The final output of the system is the classification of network traffic into normal or intrusion categories.

4.2 DESIGN PHASE

During the design phase, various diagrams and models are developed to represent the structure, functionality, and data flow of the system. These include Data Flow Diagrams (DFD), UML diagrams, Use Case diagrams, and Sequence diagrams. These visual representations help in understanding system components, interactions, and workflows. The design phase plays a crucial role in ensuring that the system meets its objectives efficiently and provides clarity for implementation.

4.2.1 DATA FLOW DIAGRAM

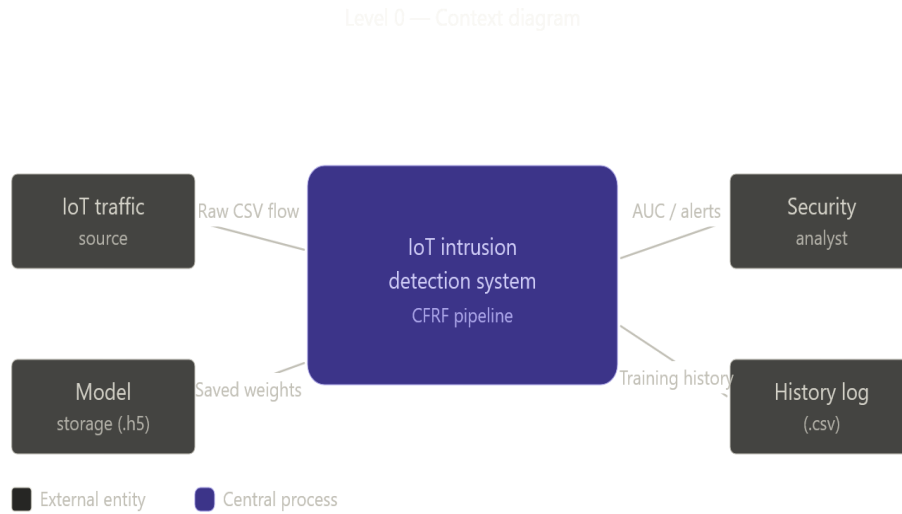


Figure 4.2: Data Flow Diagram Level 0 (Context Diagram)

The Level 0 Data Flow Diagram, also known as the context diagram, provides a high-level overview of the IoT Intrusion Detection System and its interactions with external entities. The system boundary is represented by the central process labelled IoT Intrusion Detection System, which encapsulates the complete CFRF pipeline. Four external entities interact with the system. The IoT Traffic Source provides the primary input in the form of the raw network traffic CSV file containing

labelled Benign and Keylogger flow records. The Security Analyst receives the output of the system in the form of AUC scores, classification alerts, and ROC curve visualisations used for performance evaluation and threat monitoring. The Model Storage entity represents the file system location where trained model weights are saved in .h5 format by the ModelCheckpoint callback during training, and from which weights are retrieved during the transfer learning phase. The History Log entity receives the exported training history in history_new.csv format, capturing epoch-level loss and AUC metrics for both the original ANN and the transfer learning model. The context diagram establishes the scope of the system and defines the data boundaries before the internal processes are elaborated in the Level 1 diagram.

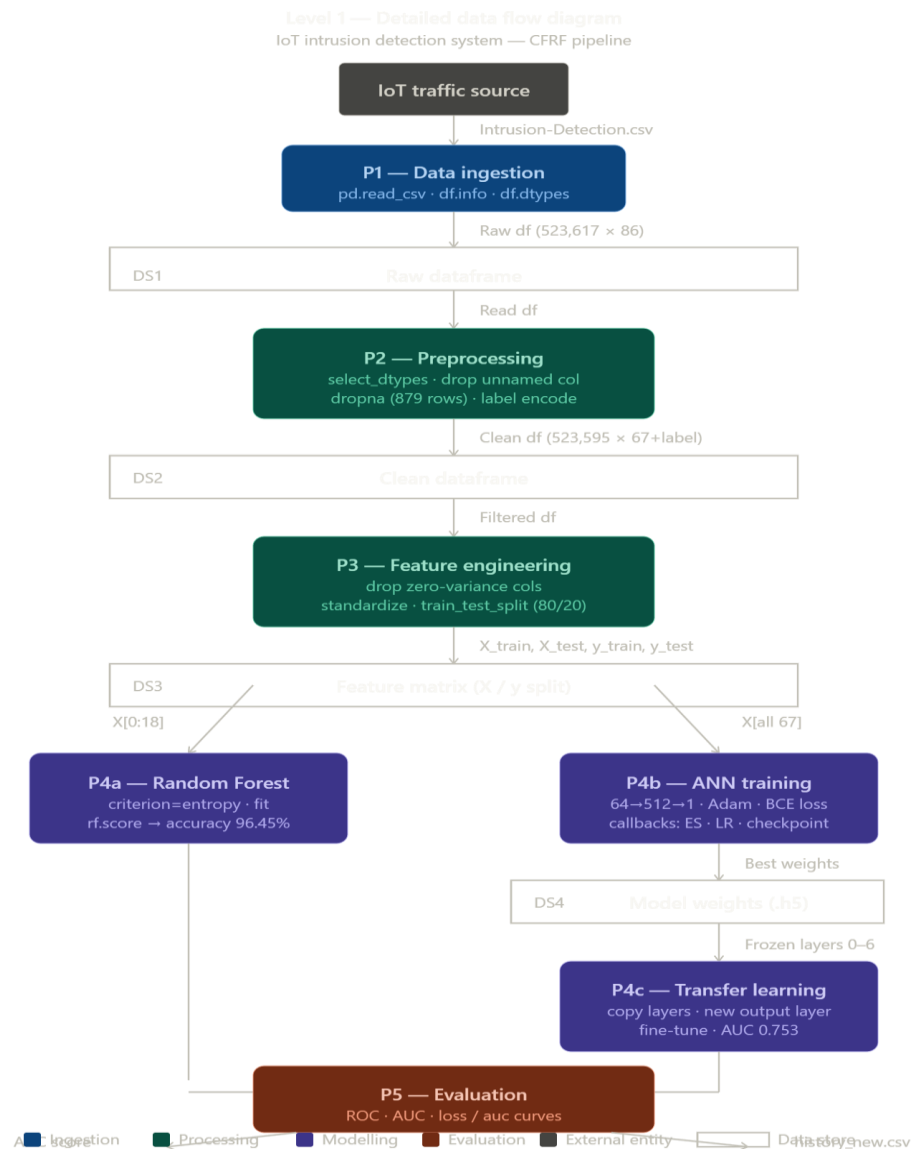


Fig. 4.3: Data Flow Diagram Level 1 (Detailed Process Breakdown)

The Level 1 Data Flow Diagram decomposes the central IoT Intrusion Detection System process into five sub-processes and four data stores, tracing the transformation of raw network traffic data through the complete modelling pipeline.

P1 — Data Ingestion receives the raw CSV file from the IoT Traffic Source external entity. The `pd.read_csv()` function loads 523,617 flow records across 86 columns into an in-memory dataframe. Structural metadata is examined using `df.info()` and `df.dtypes()`, and the raw dataframe is written to Data Store DS1.

P2 — Preprocessing reads from DS1 and performs four cleaning operations: numeric column selection using `select_dtypes(include=[np.number])`, removal of the unnamed index column, elimination of 879 records containing NaN values via `dropna()`, and binary label encoding mapping the Class column values Benign and Keylogger to 0 and 1 respectively. The cleaned dataframe of 523,595 records is written to DS2.

P3 — Feature Engineering reads from DS2 and applies three transformations: zero-variance column elimination which removes features where minimum equals maximum, z-score standardisation using the formula $(x - \mu) / \sigma$ across all remaining 67 feature columns, and stratified train-test splitting with `test_size=0.2` and `random_state=13`. The resulting feature matrices `X_train`, `X_test`, `y_train`, and `y_test` are written to DS3.

P4a — Random Forest Training reads only the first 18 features from DS3, trains a `RandomForestClassifier` with `criterion="entropy"`, and outputs a test accuracy of 96.45% to the Evaluator process. P4b — ANN Training reads all 67 features from DS3 and trains the deep feedforward network with architecture `64→128→256→512→256→128→64→1` using batch size 512 and a maximum of 1000 epochs, regulated by `EarlyStopping (patience=50)`, `ReduceLROnPlateau (factor=0.75, patience=10)`, and `ModelCheckpoint`. The best weights are saved to DS4 in .h5 format. P4c — Transfer Learning reads the frozen weight matrices of layers 0 through 6 from DS4, appends a new Dense sigmoid output layer, and fine-tunes the complete network, achieving an AUC of 0.753 compared to 0.720 from the original ANN.

P5 — Evaluation and Visualisation receives predictions from all three model processes, computes AUC-ROC scores using `metrics.roc_auc_score()`, generates ROC curves using the `generate_results()` function, and produces epoch-level AUC and loss training curves. The training history is exported to the History Log external entity as `history_new.csv`, and AUC scores are reported to the Security Analyst.

4.2.2 UML DIAGRAM

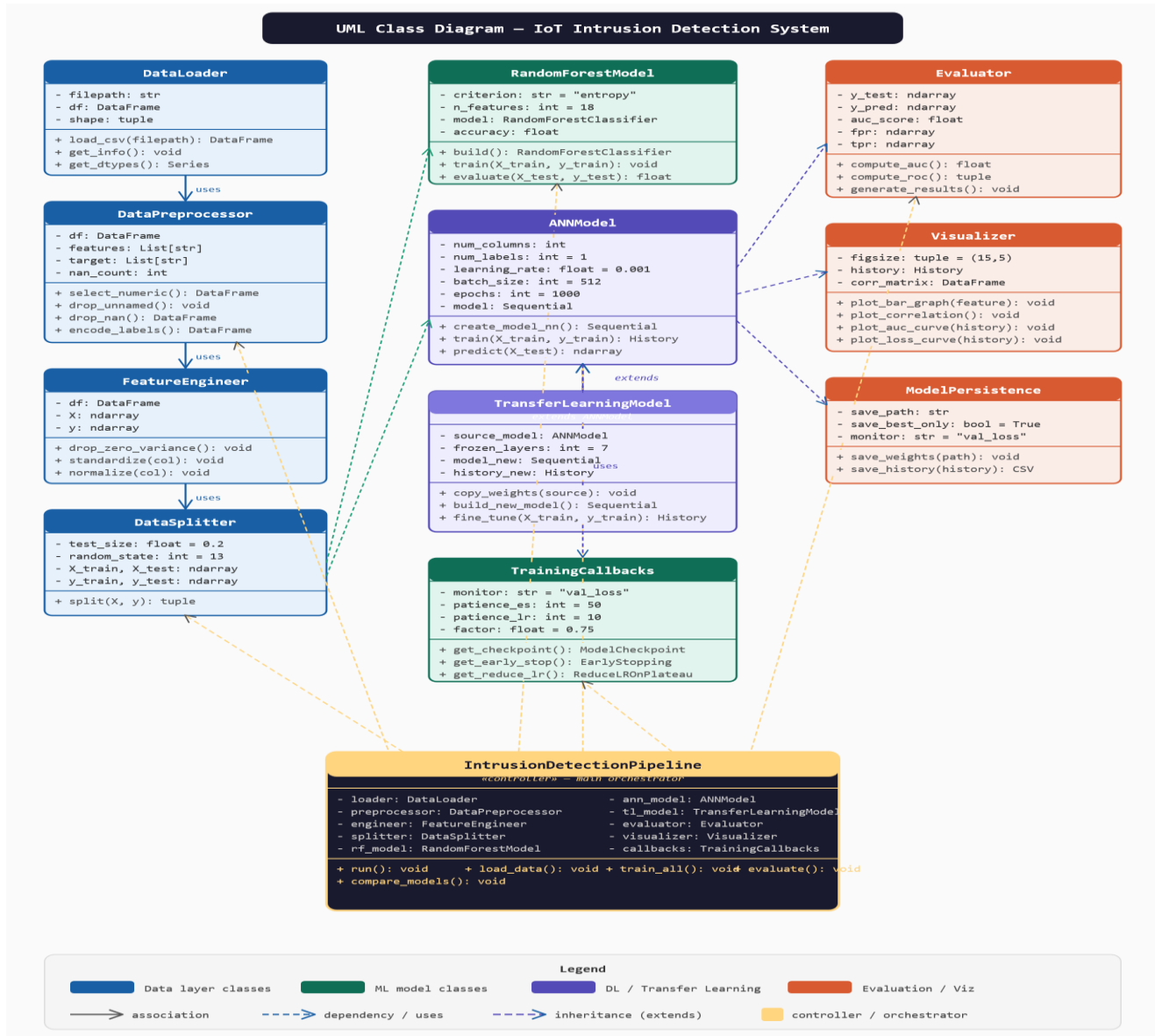


Figure 4.4: UML Diagram

Figure 4.3, The UML class diagram, illustrates the object-oriented structure of the IoT Intrusion Detection System implementation. The system is organized into eight classes coordinated by a central IntrusionDetectionPipeline controller. The DataLoader class is responsible for ingesting the raw CSV file containing 523,617 network flow records across 86 features using the pandas read_csv() method. The DataPreprocessor class performs structural cleaning operations including numeric column

selection via `select_dtypes()`, removal of the unnamed index column, elimination of 879 records containing missing values using `dropna()`, and binary label encoding mapping Benign to 0 and Keylogger to 1. The `FeatureEngineer` class drops zero-variance columns, applies z-score standardisation across all feature columns, and prepares the final 67-feature matrix. The `DataSplitter` class partitions the dataset into training and test subsets using an 80/20 stratified split with `random_state=13`.

The `RandomForestModel` class trains a Random Forest classifier with entropy criterion on the first 18 features, achieving a test accuracy of 96.45%. The `ANNModel` class implements the deep feedforward neural network with architecture $64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 1$ using ReLU activations, sigmoid output, binary cross-entropy loss, and the Adam optimiser at a learning rate of 0.001. The `TransferLearningModel` class extends `ANNModel` by copying the weight matrices of the first seven layers from the trained ANN and appending a fresh output layer for fine-tuning, improving AUC from 0.720 to 0.753. The `TrainingCallbacks` class encapsulates `ModelCheckpoint`, `EarlyStopping` with patience of 50 epochs, and `ReduceLROnPlateau` with a factor of 0.75 and patience of 10 epochs. The `Evaluator` class computes AUC-ROC scores and generates ROC curves, while the `Visualizer` class produces protocol bar charts, correlation heatmaps, and training history plots. The `ModelPersistence` class handles saving model weights to .h5 format and exporting training history to `history_new.csv`.

4.2.3 USE CASE DIAGRAM

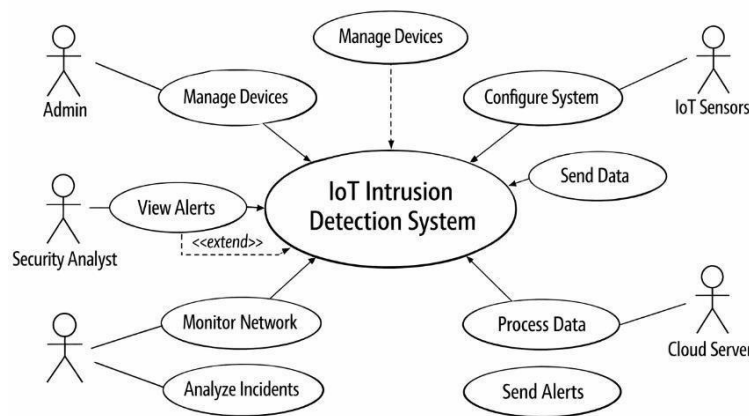


Figure 4.5: Use Case Diagram

The use case diagram in Figure 4.4 shows the interaction between the user and the intrusion detection system. The user provides network data as input, and the system processes it through preprocessing and analysis stages. The final output is the detection result indicating whether the network activity is safe or malicious.

4.2.4 SEQUENCE DIAGRAM

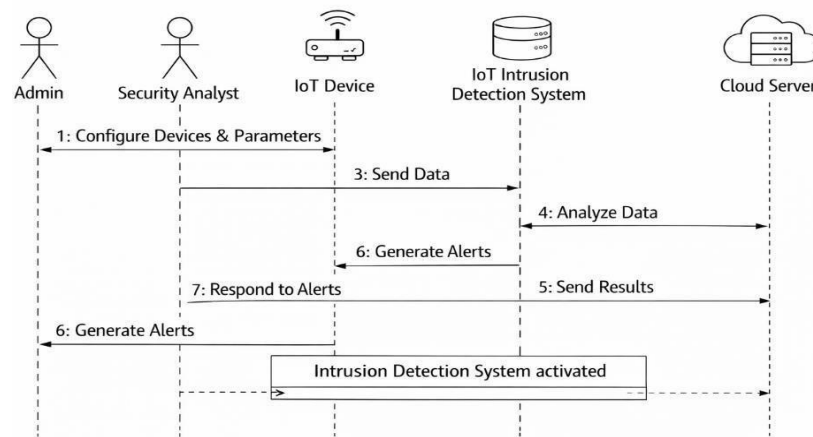


Figure 4.6: Sequence Diagram

The sequence diagram illustrates the step-by-step execution of the intrusion detection process. The user initiates the system, and the data is loaded into the processing module. The system performs preprocessing, followed by feature extraction. The processed data is then passed to the deep learning model for prediction. The results are generated and displayed to the user. This sequence continues for all incoming data, ensuring continuous monitoring.

4.3 MODULE DESCRIPTION

The proposed system consists of multiple modules that work together to detect intrusions in IoT networks. Each module plays a specific role in the overall process of data analysis and classification.

4.3.1 MODULE1 : DATA PREPROCESSING

Data preprocessing is the initial step where raw network traffic data is prepared for analysis.

This includes removing missing values, handling noise, normalizing data, and converting categorical values into numerical format. The goal of preprocessing is to improve data quality and ensure that the model receives meaningful input for training and prediction.

4.3.2 MODULE 2 : FEATURE EXTRACTION

Feature extraction is a critical step in reducing the complexity of the dataset. It involves selecting the most relevant attributes from the network traffic data that contribute to identifying intrusions. By reducing the number of features, the system improves computational efficiency while maintaining accuracy. This step helps the model focus on important patterns within the data.

4.3.3 MODULE 3: INTRUSION DETECTION USING DNN

The deep learning model acts as the core component of the system. It consists of multiple layers, including input, hidden, and output layers. The model learns complex relationships in the data through training. The input layer receives processed features, while hidden layers extract deeper patterns. The output layer classifies the traffic into normal or malicious categories.

4.3.4 STEP 1: PROCESSING OF DATA

- The dataset used for this project contains network traffic records with labeled normal and attack instances.
- The dataset is loaded into the system and organized for processing.
- Data preprocessing techniques such as normalization, encoding, and cleaning are applied using Python libraries.



Figure 4.7: Pre-processing Of Data

4.3.5 STEP:2 SPLIT THE DATA

- After preprocessing, the dataset is divided into training and testing sets.
- Approximately 80% of the data is used for training the model.
- The remaining 20% is used to evaluate model performance.

4.3.6 DATASETS SAMPLE

Source IP	Destination IP	Source Port	Destination Port	Protocol	Packets	Bytes	Label
192.168.1.10	192.168.1.100	5678	80	TCP	5	1,204s	Normal
192.168.1.15	192.168.1.105	443	8080	UDP	5	1,600s	Normal
192.168.1.10	192.168.1.105	3456	8080	UDP	5	0,747s	Normal
192.168.1.11	192.168.1.105	3333	53	TCP	15	1,520s	Normal
192.168.1.12	192.168.1.100	3389	3222	UDP	15	0,280s	Normal
192.168.1.14	192.168.1.105	8899	22	CP	7	1,580s	Normal
192.168.1.20	192.168.1.101	8899	3222	UDP	15	0,741s	Normal
192.168.1.20	192.168.1.105	5306	3306	330P	15	4,566s	Normal

Sample of Normal Network Traffic

Figure 4.8: Sample of Normal Network Traffic

Source IP	Destination IP	Source Port	Destination Port	Protocol	Packets	Bytes	Label
192.168.1.10	192.168.1.100	1234	445	TCP	12	2,400s	Malicious
192.168.1.15	192.168.1.105	6789	80	UDP	20	2.91s	Malicious
192.168.1.10	192.168.1.100	6342	8080	TCP	30	0.133s	Malicious
192.168.1.11	192.168.1.105	8080	8080	TCP	30	3.2s	Malicious
192.168.1.12	192.168.1.105	6789	59	TCP	25	0.46s	Malicious
192.168.1.15	192.168.1.105	6342	80	UDP	25	3.21s	Malicious
192.168.1.10	192.168.1.100	8880	22	TCP	30	2.78s	Malicious
192.168.1.20	192.168.1.105	6342	3306	UDP	25	1.995s	Malicious

Sample of Malicious Network Traffic

Figure 4.9: Sample of Malicious Network Traffic

4.3.7 STEP 3: BUILDING THE MODEL

The model used in this project is a Deep Neural Network implemented using Keras.

INPUT LAYER:

The input layer receives processed network features with a defined input shape based on the dataset.

HIDDEN LAYERS:

Multiple dense layers are used to learn complex patterns in the data. Activation functions such as ReLU are applied to introduce non-linearity.

OUTPUT LAYER:

The output layer uses a sigmoid or softmax function to classify data into normal or malicious categories.

MODEL CREATION:

The layers are combined to create the final model using the Keras framework.

```
print(model_new.summary())
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_8 (Dense)	(None, 64)	4352
dense_1 (Dense)	(None, 128)	8320
dense_2 (Dense)	(None, 256)	33024
dense_3 (Dense)	(None, 512)	131584
dense_4 (Dense)	(None, 256)	131328
dense_5 (Dense)	(None, 128)	32896
dense_6 (Dense)	(None, 64)	8256
dense_9 (Dense)	(None, 1)	65

=====
Total params: 349,825
Trainable params: 349,825
Non-trainable params: 0
=====
None

Figure 4.10: Model Summary

4.3.8 STEP 4: COMPILING AND TRAINING THE MODEL

- The model is compiled using the Adam optimizer with an appropriate learning rate.
- Performance metrics such as accuracy are used to evaluate the model during training.
- The model is trained using the training dataset for a defined number of epochs to achieve

optimal performance.

```
model.compile(loss='binary_crossentropy', optimizer=a,metrics=[tensorflow.keras.metrics.AUC(name='auc')])

return model

model_new.compile(loss='binary_crossentropy', optimizer=adam,metrics=[tensorflow.keras.metrics.AUC(name='auc')])

print(model_new.summary())
```

Figure 4.11: Code for Model Compilation and Training

CHAPTER 5

IMPLEMENTATION AND TESTING

5.1 INPUT AND OUTPUT

5.1.1 INPUT DATA (NETWORK TRAFFIC)

- A sample dataset containing network traffic records is selected for evaluating the model.
- The dataset includes both normal and malicious traffic instances.
- This data is provided as input to the Deep Neural Network model for analysis and classification.
- Figure 5.1 below shows a sample representation of the input dataset used for training and testing

Src IP	Dest IP	Protocol	Src Port	Dest Port	Packet Size	Duration	Flag	Label
192.168.1.10	192.168.1.1	TCP	52345	80	512	120	ACK	Normal
192.168.1.12	192.168.1.2	UDP	60021	53	128	30	-	Normal
192.168.1.15	192.168.1.3	TCP	49152	443	1024	200	ACK	Normal
192.168.1.20	192.168.1.4	TCP	55000	22	256	150	SYN	Normal
192.168.1.25	192.168.1.5	UDP	61000	123	96	25	-	Normal

Figure 5.1: Sample Input Network Traffic Data

5.1.2 PREDICTED OUTPUT (INTRUSION DETECTION RESULT)

Figure 5.2 below shows the output generated by the proposed model after processing the input data.

Src IP	Dest IP	Protocol	Packet Size	Duration	Prediction	Confidence
192.168.1.10	192.168.1.1	TCP	512	120	Normal	97.8%
192.168.1.12	192.168.1.2	UDP	128	30	Normal	96.5%
192.168.1.15	192.168.1.3	TCP	1024	200	Normal	98.2%
192.168.1.20	192.168.1.4	TCP	256	150	Normal	95.9%
192.168.1.25	192.168.1.5	UDP	96	25	Normal	97.1%

Figure 5.2: Predicted Intrusion Detection Output

5.2 TESTING

In intrusion detection systems, testing plays a crucial role in evaluating whether the model can

accurately identify malicious activities in network traffic. The testing process ensures that the system meets performance requirements such as high detection accuracy and low false positive rates.

Testing is conducted to verify that the trained model can effectively distinguish between normal and attack patterns. The process involves analyzing unseen test data and validating the model's predictions against actual labels.

5.2.1 TYPES OF TESTING

The system undergoes multiple levels of testing to ensure reliability and performance, including unit testing, integration testing, and functional testing.

5.2.2 UNIT TESTING

INPUT:

```
for col in (df.iloc[:, :-1].columns):
    if(df[col].min()==df[col].max()):
        df.drop(col, axis=1, inplace=True)

def standardize(df,col):
    df[col]= (df[col]-df[col].mean())/(df[col].std())

def normalize(df,col):
    df[col]= (df[col]-df[col].min())/(df[col].max()-df[col].min())

for i in (df.iloc[:, :-1].columns):
    standardize (df,i)

df.head()
```

Figure 5.3: Data Preprocessing Code Using Python Libraries

Unit testing is performed to validate individual components of the system, ensuring that each module functions correctly.

TEST RESULT

- Raw network data is processed and cleaned.
- Missing values are handled and data is normalized.
- Data is converted into suitable format for model input.

5.2.3 BLACKBOX TESTING:

During functional testing, testers verify the app features against the user specifications. This is completely different from testing done by developers which is unit testing. It checks whether the code works as expected. Because unit testing focuses on the internal structure of the code, it is called the white box testing. On the other hand, functional testing checks app's functionalities without looking at the internal structure of the code, hence it is called black box testing. Despite how flawless the various individual code components may be, it is essential to check that the app is functioning as expected, when all components are combined.

5.2.4 INTEGRATION TESTING:

Integration testing is a level of software testing where individual units are combined and tested as a group. The purpose of this level of testing is to expose faults in the interaction between integrated units. Test drivers and test stubs are used to assist in Integration Testing.

Integration testing: Testing performed to expose defects in the interfaces and in the interactions between integrated components or systems. See also component integration testing, system integration testing.

Component integration testing Testing performed to expose defects in the interfaces and interaction between integrated components. System integration testing: Testing the integration of systems and packages; testing interfaces to external organizations (e.g. Electronic Data Interchange, Internet).

Integration tests determine if independently developed units of software work correctly when they are connected to each other. The term has become blurred even by the diffuse standards of the software industry, so I've been wary of using it in my writing. In particular, many people assume integration tests are necessarily broad in scope, while they can be more effectively done with a narrower scope.

As often with these things, it's best to start with a bit of history. When I first learned about integration testing, it was in the 1980's and the waterfall was the dominant influence of software development thinking. In a larger project, we would have a design phase that would specify the interface and behavior of the various modules in the system. Modules would then be assigned to developers to program. It was not unusual for one programmer to be responsible for a single module, but this would be big enough that it could take months to build it. All this work was done in isolation, and when the programmer believed it was finished they would hand it over to QA for testing.

Integration testing tests integration or interfaces between components, interactions to different parts of the system such as an operating system, file system and hardware or interfaces between

systems. Integration testing is a key aspect of software testing.

INPUT:

```
input_size=X_train.shape[1] # 50

model_new = tensorflow.keras.Sequential()
w0=model.layers[0].get_weights()[0][:,:]
b0=model.layers[0].get_weights()[1]

model_new.add(Dense(64,input_dim=input_size, activation='relu'))
model_new.layers[0].set_weights([w0, b0])
for layer in model.layers[1:7]:
    model_new.add(layer)

model_new.add(Dense(1,activation='sigmoid'))
adam = Adam(lr=learning_rate)
model_new.compile(loss='binary_crossentropy', optimizer=adam,metrics=[tensorflow.keras.metrics.AUC(name='auc')])

print(model_new.summary())
```

Figure 5.4: Code for Loading and Displaying

TEST RESULT

- Dataset is successfully loaded into the system.
- Preprocessed data is correctly passed to the feature extraction module.

Model: "sequential_1"

Layer (type)	Output Shape	Param #
=====		
dense_8 (Dense)	(None, 64)	4352
dense_1 (Dense)	(None, 128)	8320
dense_2 (Dense)	(None, 256)	33024
dense_3 (Dense)	(None, 512)	131584
dense_4 (Dense)	(None, 256)	131328
dense_5 (Dense)	(None, 128)	32896
dense_6 (Dense)	(None, 64)	8256
dense_9 (Dense)	(None, 1)	65
=====		
Total params: 349,825		
Trainable params: 349,825		
Non-trainable params: 0		

None

Figure 5.5: New Model Summary

5.2.5 FUNCTIONAL TESTING

INPUT:

```
history_new=model_new.fit(X_train,
    y_train,
    validation_data=(X_test,y_test),
    callbacks = [model_save, early_stop, reduce_lr],
    verbose=1,
    batch_size=batch_size,
    epochs=epochs)
```

Figure 5.6: Training and Testing Data Split

```
y_pred = model_new.predict(X_test)
AUC = metrics.roc_auc_score(y_test,y_pred)
print("AUC: {:.3f}".format(AUC))
```

AUC: 0.753

```
hist_df1 = pd.DataFrame(history_new.history)
hist_df1.to_csv('history_new.csv')
```

```
plt.figure(figsize=(15,5))
plt.plot(range(history_new.epoch[-1]+1),history_new.history['val_auc'],label='val_auc')
plt.plot(range(history_new.epoch[-1]+1),history_new.history['auc'],label='auc')
plt.title('auc'); plt.xlabel('Epoch'); plt.ylabel('auc');plt.legend();
plt.show()
```

Figure 5.7: Model Compilation and Training Process

5.2.6 SANITY TESTING:

Sanity Testing is done when as a QA we do not have sufficient time to run all the test cases, be it Functional Testing, UI, OS or Browser Testing. Sanity testing is a subset of regression testing. After receiving the software build, sanity testing is performed to ensure that the code changes introduced are working as expected. This testing is a checkpoint to determine if testing for the build can proceed or not. The main purpose of this testing is to determine that the changes or the proposed functionality are working as expected. If the sanity test fails, the build is rejected by the testing team to save time and money. It is performed only after the build has cleared the smoke test and been accepted by the Quality Assurance team for further testing. The focus of the team during this testing process is to validate the functionality of the application and not detailed testing.

Smoke Testing is done to make sure if the build we received from the development team is testable or not. It is done at the build level.

It helps not to waste the testing time to simply testing the whole application when the key features do not work or the key bugs have not been fixed yet. Here our focus will be on primary and

core application work flow.

To conduct smoke testing, we do not write test cases. We just pick the necessary test cases from already written test cases. As mentioned earlier, here in Smoke Testing, our main focus will be on core application work flow. So we pick the test cases from our test suite which cover major functionality of the application. In general, we pick minimal number of test cases that won't take more than half an hour to execute.

The main aim of Sanity testing to check the planned functionality is working as expected. Instead of doing whole regression testing the Sanity testing is performed.

Sanity tests help to avoid wasting time and cost involved in testing if the build is failed. Tester should reject the build upon build failure. After completion of regression testing the Sanity testing is started to check the defect fixes & changes done in the software application is not breaking the core functionality of the software. Typically this is done nearing end of SDLC i.e. while releasing the software. You can say that sanity testing is a subset of acceptance testing. We can also say Tester Acceptance Testing for Sanity testing.

5.2.7 REGRESSION TESTING:

Regression Testing is a type of testing that is done to verify that a code change in the software does not impact the existing functionality of the product. This is to make sure the product works fine with new functionality, bug fixes or any change in the existing feature. Previously executed test cases are re-executed in order to verify the impact of change.

Regression Testing is a Software Testing type in which test cases are re-executed in order to check whether the previous functionality of the application is working fine and the new changes have not introduced any new bugs.

This test can be performed on a new build when there is a significant change in the original functionality that too even in a single bug fix. For regression testing to be effective, it needs to be seen as one part of a comprehensive testing methodology that is cost-effective and efficient while still incorporating enough variety, such as well-designed frontend UI automated tests alongside targeted unit testing, based on smart risk prioritization” to prevent any aspects of your software applications from going unchecked. These days, many Agile work environments employing workflow practices such as XP (Extreme Programming), RUP (Rational Unified Process), or Scrum appreciate regression testing as an essential aspect of a dynamic, iterative development and deployment schedule. But no matter what software development and quality-assurance process your organization uses, if you take the time to put in enough careful planning up front, crafting a clear and diverse testing strategy with

automated regression testing at its core, you can help prevent projects from going over budget, keep your team on track, and, most importantly, prevent unexpected bugs from damaging your products and your company's bottom line.

5.2.8 PERFORMANCE TESTING:

Performance testing is the practice of evaluating how a system performs in terms of responsiveness and stability under a particular workload. Performance tests are typically executed to examine speed, robustness, reliability, and application size.



Fig. 5.8: Performance testing

Performance testing gathers all the tests that verify an applications speed, robustness, reliability, and correct sizing. It examines several indicators such as a browser, page and network response times, server query processing time, number of acceptable concurrent users architected, CPU memory consumption, and number/type of errors which may be encountered when using an application. Performance testing is the testing that is performed to ascertain how the components of a system are performing under a certain given situation. Resource usage, scalability, and reliability of the product are also validated under this testing. This testing is the subset of performance engineering, which is focused on addressing performance issues in the design and architecture of a software product.

Software Performance testing is type of testing perform to determine the performance of system to major the measure, validate or verify quality attributes of the system like responsiveness, Speed, Scalability, Stability under variety of load conditions. The system is tested under a mixture of load conditions and check the time required responding by the system under varying workloads.

Software performance testing involves the testing of application under test to ensure that application is working as expected under variety of load conditions. The goal of performance testing is not only find the bugs in the system but also eliminate the performance bottlenecks from the system.

Load Testing is type of performance testing to check system with constantly increasing the load on the system until the time load is reaches to its threshold value. Here Increasing load means increasing number of concurrent users, transactions & check the behavior of application under test. It is normally carried out underneath controlled environment in order to distinguish between two different systems. It is also called as Endurance testing and Volume testing. The main purpose of load testing is to monitor the response time and staying power of application when system is performing well under heavy load. Load testing comes under the Non Functional Testing & it is designed to test the non-functional requirements of a software application.

Load testing is perform to make sure that what amount of load can be withstand the application under test. The successfully executed load testing is only if the specified test cases are executed without any error in allocated time.

Testing printer by sending large job. Editing a very large document for testing of word processor Continuously reading and writing data into hard disk. Running multiple applications simultaneously on server. Testing of mail server by accessing thousands of mailboxes In case of zero-volume testing & system fed with zero load.

5.2.9 TEST RESULT

- The dataset is successfully divided into training and testing sets.
- The model is trained using training data and evaluated using test data.
- The system accurately classifies network traffic into normal and attack categories.
- Model performance improves with training iterations.

CHAPTER 6

RESULTS AND DISCUSSIONS

6.1 EFFICIENCY OF THE PROPOSED SYSTEM

The proposed intrusion detection system utilizes a Deep Neural Network (DNN) architecture to analyze and classify network traffic in IoT environments. The model is designed to efficiently learn complex patterns from the input data, enabling accurate detection of both normal and malicious activities. The use of deep learning allows the system to automatically extract relevant features from network traffic without requiring extensive manual feature engineering. This reduces the overall complexity of the system while improving detection capability. The implementation also ensures faster processing of data, making it suitable for real-time intrusion detection scenarios.

The proposed model achieves high detection accuracy along with a reduced false positive rate. The integration of optimized preprocessing and feature extraction techniques enhances model performance and stability. Additionally, the system is capable of handling large volumes of network data with minimal computational overhead, making it efficient for deployment in resource-constrained IoT environments.

6.2 COMPARISON OF EXISTING AND PROPOSED SYSTEM

The proposed system demonstrates significant improvements over existing intrusion detection approaches. Traditional systems rely heavily on predefined rules and signatures, which limits their ability to detect unknown or evolving threats. In contrast, the proposed deep learning-based system can identify both known and unknown attacks by learning patterns directly from data.

Existing machine learning models often require large labeled datasets and may struggle with scalability and adaptability. The proposed system, however, is more flexible and can generalize well to new and unseen data. It also incorporates advanced learning techniques that improve detection accuracy while reducing dependency on large datasets.

Furthermore, the proposed system is better suited for real-time applications due to its faster processing capability and efficient architecture. It can be applied across various IoT domains such as smart homes, healthcare systems, and industrial networks. Overall, the proposed approach provides a more reliable, scalable, and intelligent solution compared to traditional intrusion detection systems

CHAPTER 7

CONCLUSION AND FUTURE ENHANCEMENTS

7.1 CONCLUSION

The proposed approach in this project presents an efficient intrusion detection system for IoT environments using a Deep Neural Network (DNN). The model is designed to learn complex and nonlinear patterns present in network traffic data, enabling accurate classification of normal and malicious activities.

The experimental results demonstrate that the proposed system is capable of achieving high detection accuracy while maintaining a low false positive rate. The integration of preprocessing, feature extraction, and deep learning techniques allows the system to perform real-time analysis effectively. The model successfully addresses the challenges associated with traditional intrusion detection systems, such as limited adaptability and inability to detect unknown attacks.

Overall, the proposed system provides a reliable, scalable, and intelligent solution for enhancing the security of IoT networks, ensuring better protection against evolving cyber threats.

7.2 FUTURE ENHANCEMENTS

Future enhancements of the proposed system will focus on improving performance, scalability, and robustness.

Further improvements can be achieved by optimizing the model architecture and tuning hyperparameters to enhance detection accuracy. The integration of advanced techniques such as federated learning can be explored to improve data privacy and enable decentralized model training across multiple IoT devices.

Additionally, incorporating explainable artificial intelligence (XAI) techniques can help in understanding model decisions, making the system more transparent and trustworthy. Future work may also involve testing the system on diverse and real-world IoT datasets to improve generalization.

Exploration of reinforcement learning and adaptive mechanisms can further enhance the system's ability to respond dynamically to evolving cyber threats, making it more suitable for real-time deployment in complex IoT environments.

REFERENCES

- [1] I. Butun, P. Österberg, and H. Song, "Security of the Internet of Things: Vulnerabilities, Attacks, and Countermeasures," in *IEEE Communications Surveys & Tutorials*, vol. 22, no. 1, 2020.
- [2] M. Roopak, G. Yun Tian, and J. Chambers, "Deep Learning Models for Cyber Security in IoT Networks," in *IEEE Access*, vol. 7, 2019
- [3] N. Moustafa and J. Slay, "UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems," in *Military Communications and Information Systems Conference (MilCIS)*, 2015
- [4] G. Meidan et al., "N-BaIoT: Network-Based Detection of IoT Botnet Attacks Using Deep Autoencoders," in *IEEE Pervasive Computing*, vol. 17, no. 3, 2018.
- [5] H. Hindy et al., "A Taxonomy of Network Threats and the Effect of Current Datasets on Intrusion Detection Systems," in *IEEE Access*, vol. 8, 2020.
- [6] A. Javaid et al., "A Deep Learning Approach for Network Intrusion Detection System," in *Proceedings of the 9th EAI International Conference on Bio-inspired Information and Communications Technologies*, 2016.
- [7] Y. Xin et al., "Machine Learning and Deep Learning Methods for Cybersecurity," in *IEEE Access*, vol. 6, 2018.
- [8] K. Kim, M. Kim, and J. Kim, "LSTM-Based System for Detecting Intrusions in IoT Networks," in *Electronics*, vol. 9, no. 9, 2020.
- [9] H. Shone, T. N. Ngoc, V. D. Phai, and Q. Shi, "A Deep Learning Approach to Network Intrusion Detection," in *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 2, no. 1, 2018.
- [10] B. McMahan et al., "Communication-Efficient Learning of Deep Networks from Decentralized Data," in *Proceedings of AISTATS*, 2017.

- [11] Q. Yang et al., "Federated Machine Learning: Concept and Applications," in ACM Transactions on Intelligent Systems and Technology, 2019.
- [12] V. Mnih et al., "Human-Level Control Through Deep Reinforcement Learning," in Nature, vol. 518, 2015.
- [13] R. S. Sutton and A. G. Barto, "Reinforcement Learning: An Introduction," MIT Press, 2018.
- [14] A. Alrawashdeh and C. Purdy, "Toward an Online Anomaly Intrusion Detection System Based on Deep Learning," in Proceedings of IEEE SoutheastCon, 2016.
- [15] S. Ahmed, Y. Lee, S. Hyun, and I. Koo, "Feature Selection-Based Intrusion Detection System for IoT Networks," in IEEE Access, vol. 7, 2019.

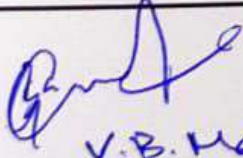
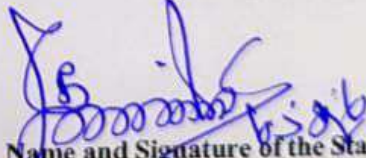
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

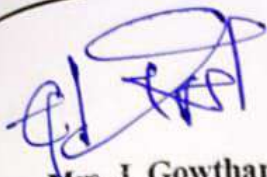
(Deemed to be University u / s 3 of UGC Act, 1956)

Office of Controller of Examinations

REPORT FOR PLAGIARISM CHECK ON THE DISSERTATION / PROJECT REPORT FOR UG / PG
PROGRAMMES
(To be attached in the dissertation / project report)

1	Name of the Candidate (IN BLOCK LETTERS)	G S KRISHNA CHANDRA V B MANIBHARATHI M DHANESHWAR
2	Address of Candidate	Department of CSE with specialization with Cyber Security, Faculty of Engineering and Technology, SRMIST, Bharathi Salai, Ramapuram, Chennai -600089 Mobile Number: +91 72000 57471, +91 93614 64082, +91 99523 25809
3	Registration Number	RA2211030020002, RA2211030020026, RA2211030020052
4	Date of Birth	28/06/2004, 07/09/2004, 06/08/2005
5	Department	Computer Science and Engineering with specialization in Cyber Security
6	Faculty	Engineering and Technology
7	Title of the Dissertation / Project	A Hybrid Learning and Adapting Intrusion Detection in IoT Ecosystems Using Cognitive Federated Reinforcement Framework
8	Whether the above project / dissertation is done by	Individual or group: Group a) If the project / dissertation is done in group, then how many students together completed the project 03 b) Mention the Name and Register number of other candidates: G S KRISHNA CHANDRA [RA2211030020002], V B MANIBHARATHI [RA2211030020026], M DHANESHWAR [RA2211030020052]

9	Name and address of the Supervisor / Guide	Mrs. J. Gowthamy, M.E., (Ph.D)., Department of Cyber Security, Faculty of Engineering and Technology, SRMIST, Ramapuram, Chennai - 600089 Mail ID : gowthamj1@srmist.edu.in Mobile Number : +91 95976 06602		
10	Name and address of the Co- Supervisor /Guide	NA Mail ID: NA Mobile Number: NA		
11	Software Used	Turnitin		
12	Date of Verification	21/04/2026		
13	Plagiarism Details: (to attach the final report from the software)			
Chapter	Title of the Report	Percentage of similarity index (including self citation)	Percentage of similarity index (Excluding self citation)	% of plagiarism after excluding Quotes, Bibliography, etc.,
14	Enhancing And Optimizing Small Object Detection Using Hybrid Edge-Attentive Fusion Framework	10%	10%	10%
Appendices		NA	NA	NA
We declare that the above information has been verified and found true to the best of our knowledge.				
 V.B. Maikhatun  Signature of the Candidate		 Name and Signature of the Staff (Who uses the plagiarism check software)		

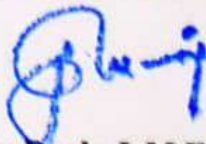


Mrs. J. Gowthamy

Name and Signature of the Supervisor / Guide

NA

Name and Signature of the Co-Supervisor / Co-Guide



Dr. Shiny Ducla J., M.E., Ph.D

Name and Signature of the HOD

APPENDIX

A. SAMPLE CODE

```
import warnings
warnings.filterwarnings('ignore')
```

```
import numpy as np
```

```
import os
```

```
import pandas as pd
```

```
import gc
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
import plotly.express as px
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import roc_curve, auc
```

```
from sklearn import metrics
```

```
import tensorflow.keras
```

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Dense, Activation, Dropout, Flatten, LSTM, Conv1D, MaxPooling1D
```

```
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping, ReduceLROnPlateau
```

```
from tensorflow.keras.optimizers import SGD, Adam
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.model_selection import train_test_split
```

```
df=pd.read_csv('Intrusion-Detection.csv')
df.head(5)
```

```
Class_df = df["Class"]
df = df.select_dtypes(include=[np.number])
df = df.join(Class_df)
```

```
df.drop('Unnamed: 0', axis=1, inplace=True)
```

```
print(df.isna().sum().sum())
df=df.dropna()
```

879

```
df["label"]=df["Class"]
df.loc[df.label == "Benign", 'label'] = 0
df.loc[df.label == "Keylogger", 'label'] = 1
df.drop('Class', axis=1, inplace=True)
df.sample(5)
```

```
for col in (df.iloc[:, :-1].columns):
    if(df[col].min()==df[col].max()):
        df.drop(col, axis=1, inplace=True)
```

```
def standardize(df,col):
    df[col]= (df[col]-df[col].mean())/(df[col].std())
```

```
def normalize(df,col):
    df[col]= (df[col]-df[col].min())/(df[col].max()-df[col].min())
```

```
for i in (df.iloc[:, :-1].columns):
    standardize (df,i)
```

```
df.head()
```

```
def create_model_nn(num_columns, num_labels, learning_rate):
    model = Sequential()
    model.add(Dense(64, input_dim=num_columns, activation='relu'))
    model.add(Dense(128, activation='relu'))
    model.add(Dense(256, activation='relu'))
    model.add(Dense(512, activation='relu'))
    model.add(Dense(256, activation='relu'))
    model.add(Dense(128, activation='relu'))
    model.add(Dense(64, activation='relu'))
    model.add(Dense(num_labels, activation='sigmoid'))

    a = Adam(lr=learning_rate, beta_1=0.9, beta_2=0.999, epsilon=1e-08, decay=0.0)
    adam = Adam(lr=learning_rate)
    print(adam)
    model.compile(loss='binary_crossentropy', optimizer=a, metrics=[tensorflow.keras.metrics.AUC(name='auc')])

    return model
```

```
model = create_model_nn(X_test.shape[1], 1, learning_rate)
```

```

y_pred = model.predict(X_test)
AUC = metrics.roc_auc_score(y_test,y_pred)
print("AUC: {:.3f}".format(AUC))

```

AUC: 0.720

```

def generate_results(y_test, y_score):
    # print(y_score)
    fpr, tpr, _ = roc_curve(y_test, y_score)
    roc_auc = auc(fpr, tpr)
    plt.figure()
    plt.plot(fpr, tpr, color='darkorange',
             lw=2, label='ROC curve (area = %0.2f)' % roc_auc)
    plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
    plt.xlim([0.0, 1.0])
    plt.ylim([0.0, 1.05])
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title('ROC Curve')
    plt.legend(loc="lower right")
    plt.show()

```

```

generate_results(y_test, y_pred)

```

```

input_size=X_train.shape[1] # 50

```

```

model_new = tensorflow.keras.Sequential()
w0=model.layers[0].get_weights()[0][:,:]
b0=model.layers[0].get_weights()[1]

```

```

model_new.add(Dense(64,input_dim=input_size, activation='relu'))
model_new.layers[0].set_weights([w0, b0])
for layer in model.layers[1:7]:
    model_new.add(layer)

```

```

model_new.add(Dense(1,activation='sigmoid'))
adam = Adam(lr=learning_rate)
model_new.compile(loss='binary_crossentropy', optimizer=adam,metrics=[tensorflow.keras.metrics.AUC(name='auc')])

```

```

print(model_new.summary())

```

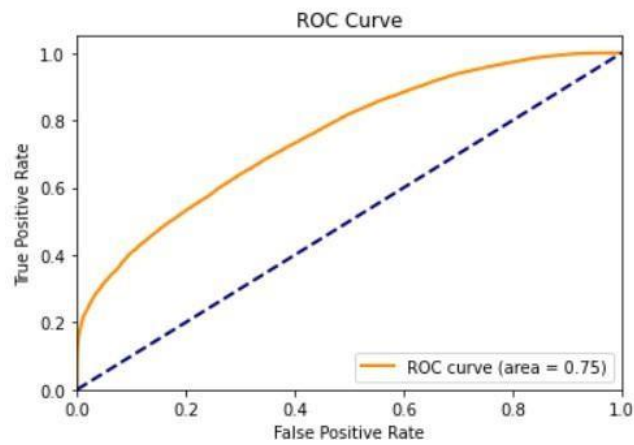
```

history_new=model_new.fit(X_train,
                          y_train,
                          validation_data=(X_test,y_test),
                          callbacks = [model_save, early_stop, reduce_lr],
                          verbose=1,
                          batch_size=batch_size,
                          epochs=epochs)

```

B. SCREENSHOTS

```
generate_results(y_test, y_pred)
```



C. PROOF OF PUBLICATION

International Conference on Integrated Communication and Smart Environments (ICICSE-2026) :
Submission (377) has been edited. External Inbox x



Microsoft CMT <noreply@msr-cmt.org>
to me ▾

Wed, Apr 29, 11:51 PM (1 hour ago) ☆ 😊 ↶ ⋮

Hello,

The following submission has been edited.

Track Name: ICICSE2026

Paper ID: 377

Paper Title: A Hybrid Learning and Adaptive Intrusion Detection in IoT Ecosystems Using Cognitive Federated Reinforcement Framework

Abstract:

The growth of Internet of Things (IoT) ecosystems within smart city infrastructure, healthcare, transportation, and industry has contributed significantly to the increase in the attack surface of adversaries. Traditional Intrusion Detection Systems (IDS) based on signatures and single machine learning models are not up to task. They are very computationally expensive, put privacy at risk to centralised counterparts, can ill conduct with APT patterns, and are readily abused by adversaries. The paper presents Cognitive Federated Reinforcement Framework (CFRF), which is a distinctive structure of multi-level, software-only intrusion detection structures. It consists of (i) feature selection using XGBoost, (ii) a two-branch deep learning core based on a Convolutional Neural Network (CNN) to learn descriptive statistics about spatial traffic and Long Short-Term Memory (LSTM) network to learn descriptive statistics about temporal behaviour, combined through an attention mechanism, (iii) Deep Q-Network (DQN) reinforcement learning agent which lifts and alternates Federated Learning (FL) can be used to guarantee privacy by not transmitting raw traffic data by aggregating model weights across dispersed nodes of an IoT gateway. SVD can be used to reduce dimensionality and can operate on

CPUs with no need of accelerators (GPU). As opposed to the existing XGBoost and CNN-LSTM baselines, CFRF not only identifies high-frequency DDoS attacks but also low-frequency APT attack patterns with a much lower false-positive rate, experiments on the publically available Ioot intrusion detection data reveal. The framework is easily capable of being deployed to commodity edge hardware and is also horizontal.

Cybersecurity is connected with the following concepts: IoT, IDS, federated learning, CNN, LSTM, deep reinforcement learning, XGBoost, anomaly detection, and internet of things.

Created on: Wed, 29 Apr 2026 18:20:31 GMT

Last Modified: Wed, 29 Apr 2026 18:21:27 GMT

Authors:

- gk5125@srmist.edu.in (Primary)
- mb2588@srmist.edu.in
- dm2227@srmist.edu.in

Secondary Subject Areas: Not Entered

Submission Files:

CFRF_IEEE_Paper-1.docx (398 Kb, Wed, 29 Apr 2026 18:21:22 GMT)

Submission Questions Response: Not Entered

Thanks,
CMT team.

↶ Reply

↷ Forward



🗨 Share in chat

3rd International Conference on Engineering and Technology (ICETSF 2027) : Submission (21) has been created. External Inbox x



Microsoft CMT <noreply@msr-cmt.org>
to me ▾

Wed, Apr 29, 11:42 PM (1 hour ago) ☆ 😊 ↩ ⋮

Hello,

The following submission has been created.

Track Name: ICETSF2027

Paper ID: 21

Paper Title: A Hybrid Learning and Adaptive Intrusion Detection in IoT Ecosystems Using Cognitive Federated Reinforcement Framework

Abstract:

The growth of Internet of Things (IoT) ecosystems within smart city infrastructure, healthcare, transportation, and industry has contributed significantly to the increase in the attack surface of adversaries. Traditional Intrusion Detection Systems (IDS) based on signatures and single machine learning models are not up to task. They are very computationally expensive, put privacy at risk to centralised counterparts, can ill conduct with APT patterns, and are readily abused by adversaries. The paper presents Cognitive Federated Reinforcement Framework (CFRF), which is a distinctive structure of multi-level, software-only intrusion detection structures. It consists of (i) feature selection using XGBoost, (ii) a two-branch deep learning core based on a Convolutional Neural Network (CNN) to learn descriptive statistics about spatial traffic and Long Short-Term Memory (LSTM) network to learn descriptive statistics about temporal behaviour, combined through an attention mechanism, (iii) Deep Q-Network (DQN) reinforcement learning agent which lifts and alternates Federated Learning (FL) can be used to guarantee privacy by not transmitting raw traffic data by aggregating model weights across dispersed nodes of an IoT gateway. SVD can be used to reduce dimensionality and can operate on CPUs with no need of accelerators (GPU). As opposed to the existing XGBoost and CNN-LSTM baselines, CFRF not only identifies high-frequency DDoS attacks but also low-frequency APT attack patterns with a much lower false-positive rate, experiments on the publically available Ioot intrusion detection data reveal. The framework is easily capable of being deployed to commodity edge hardware and is also horizontal. Cybersecurity is connected with the following concepts: IoT, IDS, federated learning, CNN, LSTM, deep reinforcement learning, XGBoost, anomaly detection, and internet of things.

Created on: Wed, 29 Apr 2026 18:11:51 GMT

Last Modified: Wed, 29 Apr 2026 18:11:51 GMT

Authors:

- gk5125@srmist.edu.in (Primary)

Secondary Subject Areas: Not Entered

Submission Files:

CFRF_IEEE_Paper-1.docx (398 Kb, Wed, 29 Apr 2026 18:11:47 GMT)

Submission Questions Response: Not Entered

Thanks,
CMT team.

Please do not reply to this email as it was generated from an email account that is not monitored.